Repository Description: Cards (Blackjack + Texas Hold'em Strategy Engines)

Written 2026-10-06 from a read of the code, docs and baselines. The test suites and the app were not run as part of writing this.

This repo holds two card-game strategy engines behind one Flask app: a finished **blackjack** engine and a newer, strictly additive **Texas Hold'em** engine. Both compute strategy from first principles and use published results only as known-answer tests.

Blackjack engine

Every number is a function of a frozen `RuleSet` dataclass in `src/engine/rules.py`. It covers decks, S17/H17, double-after-split, doubling restrictions, surrender, blackjack payout, resplit rules, peek/no-peek and penetration. Its `key()` (e.g. `6d-s17-das-ls-p75`) names every cached artifact.

The pipeline runs in layers:

Layer	Files	What it does
Mechanics	`src/engine/`	Cards, shoe with cut card, hands, dealer draw, and a round state machine in `game.py`. Shoe composition is a 10-bucket rank vector with T/J/Q/K collapsed.
Exact solver	`src/strategy/dealer_probs.py`, `ev_solver.py`	Dealer outcome distribution (conditioned on "no blackjack" after a peek) and memoised recursive EV of stand, hit, double, split and surrender.
Strategy grid	`src/strategy/grid_builder.py`, `basic_strategy.py`, `exact_edge.py`	Solves every hard, soft and pair cell, caches the grid as JSON, serves runtime lookups, and computes an exact house edge.
Counting	`src/counting/`	Hi-Lo, KO, Hi-Opt, Omega II, Zen and Red 7; running and true count (floor for betting, nearest for playing).
Deviations	`src/strategy/index_builder.py`, `deviations.py`	Derives Illustrious 18, Fab 4 and insurance indices by re-solving cells against count-biased shoes.
Simulation	`src/simulation/`, `src/counting/edge_model.py`	Monte Carlo batteries with standard errors, N_0, risk of ruin, SCORE, and a linear fit of edge against true count.
Bankroll	`src/bankroll/`	Fractional (default half) Kelly sizing, a discrete bet ramp, drawdown simulation and risk profile.

Cached grids are validated by `src/utils/cache_manager.py` using a hash of the solver source plus the ruleset, and rebuilt on any mismatch.

Shipped data and baselines

- `data/strategy/` holds 32 solved grids: {2, 4, 6, 8} decks × S17/H17 × DAS/no-DAS × late/no surrender.
- `data/indices/` holds one pre-built index table, 6-deck Hi-Lo.
- `data/baselines/house_edge_baseline.json` locks the exact house edge against published anchors within ±0.02%:

Decks	Exact edge	Published anchor
2	0.1727%	0.19%
4	0.3439%	0.35%
6	0.4002%	0.40%
8	0.4284%	0.43%

- The locked edge model (6-deck Hi-Lo, 20M hands) is `edge(TC) = -0.00326 + 0.00479·TC`, with R^2 = 0.979, according to `.agents/project-overview.md`.

Poker engine

Everything lives under `src/poker/` and is keyed by `GameSpec` in `spec.py`, the analogue of `RuleSet` (players, stack depth, blinds, antes, rake, bet sizes). Rake is applied in exactly one place.

- **Evaluation and equity:** `evaluator.py` is an exact 5/6/7-card evaluator over 7,462 rank classes. `equity.py` enumerates exactly where tractable and uses Monte Carlo with standard errors otherwise. `ranges.py` handles the 169-hand grid notation, weighted combos and card removal.
- **Solving:** `tree.py` builds the betting tree, `abstraction.py` buckets hands, and `cfr.py` implements vanilla CFR, CFR+, discounted CFR and external-sampling MCCFR.
- **Validation:** `exploit.py` measures best-response exploitability. `toy.py` provides Kuhn and Leduc poker with an independent sequence-form LP solution to check CFR against.
- **Applications:** `pushfold.py` and `icm.py` solve jam/fold equilibria and Malmuth-Harville ICM. `advisor.py` looks up preflop action frequencies, and `bankroll.py` covers risk of ruin and win-rate confidence intervals.
- **Scripts:** `scripts/poker/` builds the evaluator tables, the 169×169 preflop equity matrix, preflop solves, and bounded single-spot postflop solves.

Poker data is about 97 MB under `data/poker/`. The evaluator tables and raw solves are gitignored; the equity matrix, ranges, strategies and baselines are meant to be committed.

Limitations the repo records about itself

- Leduc game value matches the LP to 2e-5, but exploitability reached 0.0092 against a 1e-3 target.
- The two heads-up preflop solves have high abstraction exploitability: about 2,272 mbb at 100bb and 483 mbb at 10bb.
- Preflop terminal values rely on an equity-realisation approximation, which the build plan calls its weakest link.
- Scope is heads-up, preflop plus bounded single-spot postflop; full-tree solving is out of scope.

Web app

`app.py` serves both single-page apps and the blackjack API; the poker API is a Flask Blueprint in `src/poker/api.py`.

	Blackjack	Poker
Page	`/` → `static/index.html`	`/poker` → `static/poker.html`
Tabs	Play, Strategy Grid, Counting, Bankroll, Simulator	Equity, Ranges, Preflop, Push/Fold & ICM, Solver, Bankroll, Trainer
API	`/api/rules`, `/grid`, `/advise`, `/count`, `/stake`, `/simulate`, `/game/{new,deal,action}`	`/api/poker/spec`, `/equity`, `/range/parse`, `/range/equity`, `/preflop`, `/advise`, `/pushfold`, `/icm`, `/solve`, `/bankroll`

The frontend is vanilla JS with a dark glassmorphism style, with separate bundles per game. `main.py` is a CLI that prints a blackjack grid.

Tests and CI

- **Unit tests:** roughly 86 blackjack test functions in `tests/` and 208 poker ones in `tests/poker/`.
- **Regression gates:** `tests/regression_test.py` and `tests/poker/regression_test.py` recompute results and compare them with the locked baselines. Each has a `--fast` mode for CI.
- **CI:** `.github/workflows/regression.yml` runs blackjack and poker as two separate jobs, so a failure names the engine that broke.

Commands:

```
py -m pytest tests/ --ignore=tests/regression_test.py -q   # unit tests
py tests/regression_test.py --fast                         # blackjack gate, CI mode
py tests/poker/regression_test.py --fast                   # poker gate, CI mode
py app.py                                                  # app + API on http://127.0.0.1:5000
```

Documentation and process

- `.agents/` holds the working instructions, project overview, coding standards and per-topic skills (mirrored in `.claude/skills/`).
- `markdowns/blackjack/blackjack_build_plan_v1.md` (12 phases) and `markdowns/poker/poker_build_plan_v1.md` (14 phases) record the plan, per-phase results and bugs found.
- `docs/system_guide.md` and `docs/poker_guide.md` explain the reasoning behind the solvers.
- The blackjack engine is frozen behind its baselines, and poker work is not allowed to modify it.

Things that are out of date or unusual

- **Most of the project is not committed.** Git has 20 commits covering the blackjack engine through validation. The untracked set includes `app.py`, `static/`, `src/counting/`, `src/bankroll/`, `src/poker/`, `docs/`, `data/baselines/`, the CI workflow, and most tests.
- **`README.md` is stale.** It still says "Pre-implementation", while both build plans mark every phase complete.
- **The project overview understates poker.** It says poker is at "Phase 0 only", but the poker tracking table shows all 14 phases done.
- **Coding standards reference a missing file.** They mention a `strategy_baseline.json` that was never created; the grid is checked directly against the Appendix A fixture instead.
- **Python versions differ.** CI runs 3.12, while the coding standards say 3.14+.